

Experiment 4

k-Nearest Neighbors (KNN)

Aim:

To implement the k-Nearest Neighbours (KNN) algorithm for classifying the Iris dataset and evaluate its performance using standard classification metrics.

Theory:

The k-Nearest Neighbours (KNN) algorithm is a supervised machine learning algorithm used for classification and regression. It is a non-parametric method, as it does not assume any prior distribution of the data. In KNN, classification is performed based on the similarity between data samples, which is commonly measured using distance metrics such as Euclidean distance.

The nearest neighbour classification concept was introduced by Fix and Hodges and later extended by Cover and Hart. In this method, a test sample is assigned a class label by identifying the k nearest training samples in the feature space and applying majority voting. The value of k represents the number of nearest neighbours considered for classification.

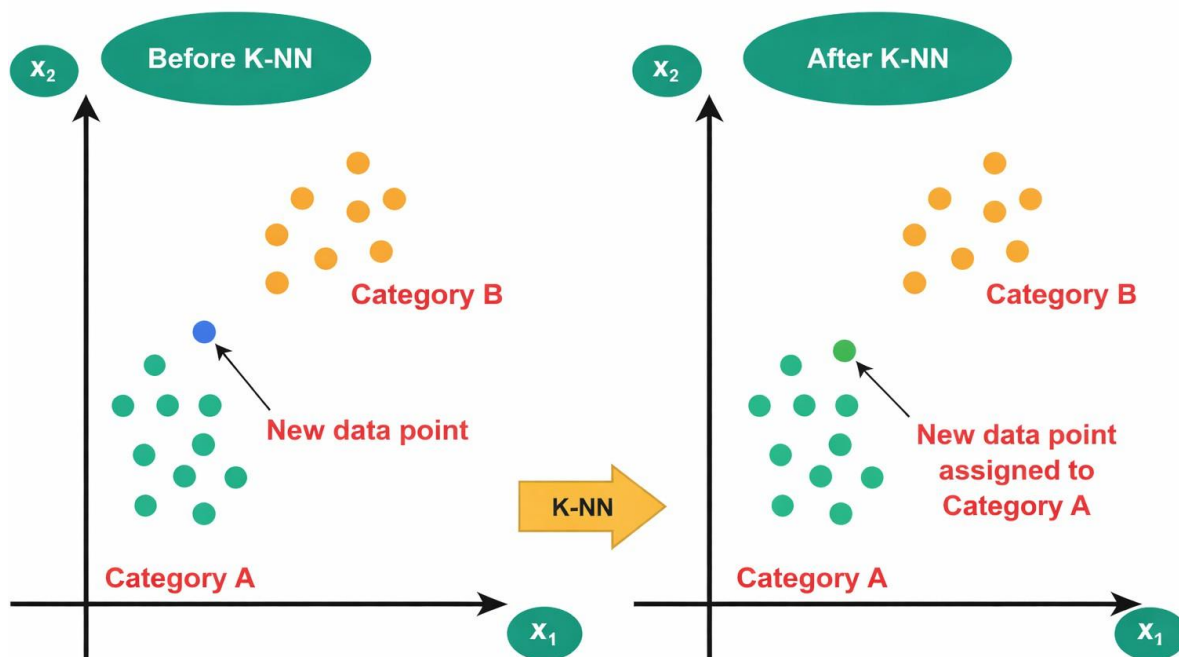
KNN does not involve an explicit training phase. Instead, it stores the entire training dataset and performs computation during the testing phase. Hence, it is referred to as a lazy learning or instance based learning algorithm. The choice of k significantly affects classifier performance. Smaller values of k may result in overfitting, while larger values of k may lead to underfitting.

Since KNN is based on distance calculations, feature scaling is important to ensure that all features contribute equally. Without proper scaling, features with larger numerical ranges may dominate the distance computation.

Merits and Demerits of k-Nearest Neighbours (KNN)

Merits of KNN	Demerits of KNN
Simple and easy to understand	High computational cost during prediction
Does not require an explicit training phase	Requires large memory to store training data
Makes no assumption about data distribution	Highly sensitive to feature scaling
Works well for small and well-separated datasets	Performance degrades in high-dimensional data
Can be used for both classification and regression	Sensitive to noise and outliers

The below figure illustrates how K-NN assigns a class to a new data point by considering the majority class among its nearest neighbours.



Code Section:

1. Importing Required Libraries:

```
# Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
print ("Imported Analysis and Plotting libraries")

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import label_binarize
from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
```

```
confusion_matrix
)
print ("Imported Sklearn Modules")
```

2. Loading and Reading the Dataset:

```
# Loading and Reading Data
iris = load_iris () # returns data in numpy arrays
X = iris.data
y = iris.target
print ("Extracted Feature and Target Arrays")

names = iris.feature_names # extracting feature names
target_names = iris.target_names # extracting target names
iris_df = pd.DataFrame(X, columns=names) # making a dataframe with column names
iris_df["label"] = [target_names[i] for i in y]
iris_df.head()
```

3. Exploratory Data Analysis:

```
# Data Analysis

iris_df.info () # Data Overview

iris_df["label"].value_counts () # Target counts

iris_df.describe() # Dataset description

iris_df.isnull().sum () # Finding any null values
```

4. Feature Distribution Visualization:

```
fig, axes = plt.subplots(2, 2, figsize= (10, 8))
for ax, col in zip(axes.ravel(), names):
    sns.histplot(
        data=iris_df, x=col, hue="label",
        kde=True, element="step", ax=ax
    )
    ax.set_xlabel (col, fontweight='bold')
    ax.set_ylabel ("Count", fontweight='bold')

plt.suptitle("Feature Distributions by Class", y=1.02, fontweight='bold')
plt.tight_layout ()
plt.show()
```

5. Feature Correlation Heatmap:

```
plt.figure(figsize= (10, 8))
ax = sns.heatmap(
    iris_df.iloc[:4].corr (),
    annot=True,
```

```
cmap="crest",
fmt=".2f",
linewidths=0.7,
annot_kws= {"color": "black"} # Force annotation text to be black
)
plt.title("Feature Correlation Heatmap", fontweight='bold')
plt.show()
```

6. Data Encoding:

Data Encoding

```
iris_df["target"] = y # we already have encoded targets from sklearn API call
iris_df.head()
```

7. Data Preprocessing and Train–Test Split:

Data Preprocessing

```
X = iris_df.iloc[:4]. values
```

```
y = iris_df["target"]. values
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, # Important as it decides the behaviour of model
    random_state=42
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

8. Feature Scaling:

Scaling the values

```
scaler = StandardScaler ()
```

```
scaler
```

```
X_train_s = scaler.fit_transform(X_train)
```

```
X_test_s = scaler.transform(X_test)
```

```
print ("Values scaled with Standard Scaler")
```

```
X_train [:5]
```

```
X_train_s [:5]
```

9. Label Binarization for ROC-AUC:

Binarizing encoded labels for ROC-AUC metric calculation

```
y_test_bin = label_binarize (y_test, classes= [0,1,2])
```

```
y_test_bin [:5]
```

10. Model Training:

Model Training

```
print (" | | Cell in development | |")
```

```
model = KNeighborsClassifier(n_neighbors=10) # k = 10
```

```
model.fit(X_train_s, y_train)
print (" | | Animation Here | | ")
```

11. Model Evaluation and Prediction

```
# Model Evaluation
y_pred = model.predict(X_test_s)
y_pred

y_prob = model.predict_proba(X_test_s)
y_prob[:5]

print ("\nClassification Report (k=10):")
print (classification_report (
    y_test,
    y_pred,
    target_names=target_names,
    digits=4
))
```

12. Confusion Matrix Visualization:

```
plt.figure(figsize= (7, 5))
sns.heatmap(
    confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Greens",
    xticklabels=target_names, yticklabels=target_names
)
plt.xlabel("Predicted", fontweight='bold')
plt.ylabel("True", fontweight='bold')
plt.title("Confusion Matrix Heatmap (KNN, k=10)", fontweight='bold')
plt.tight_layout ()
plt.show()
```

13. Final Evaluation Metrics:

```
# Final Metrics and how visualization of learning
print (f"Accuracy: {accuracy_score (y_test, y_pred)}")
Precision: {precision_score (y_test, y_pred, average="macro")}
Recall: {recall_score (y_test, y_pred, average="macro")}
F1: {f1_score (y_test, y_pred, average="macro")}
ROC: {roc_auc_score(y_test_bin, y_prob, multi_class="ovr", average="macro")}
""")

Print ("\n | | Cell in development | | ")
Print (" | | Animation here | | ")
```

Results:

1. Dataset Summary:

The Iris dataset contains **150 samples** with **four numerical features**: sepal length, sepal width, petal length, and petal width.

The dataset is perfectly balanced with **50 samples each** for *Setosa*, *Versicolor*, and *Virginica*.

No missing values were found in the dataset.

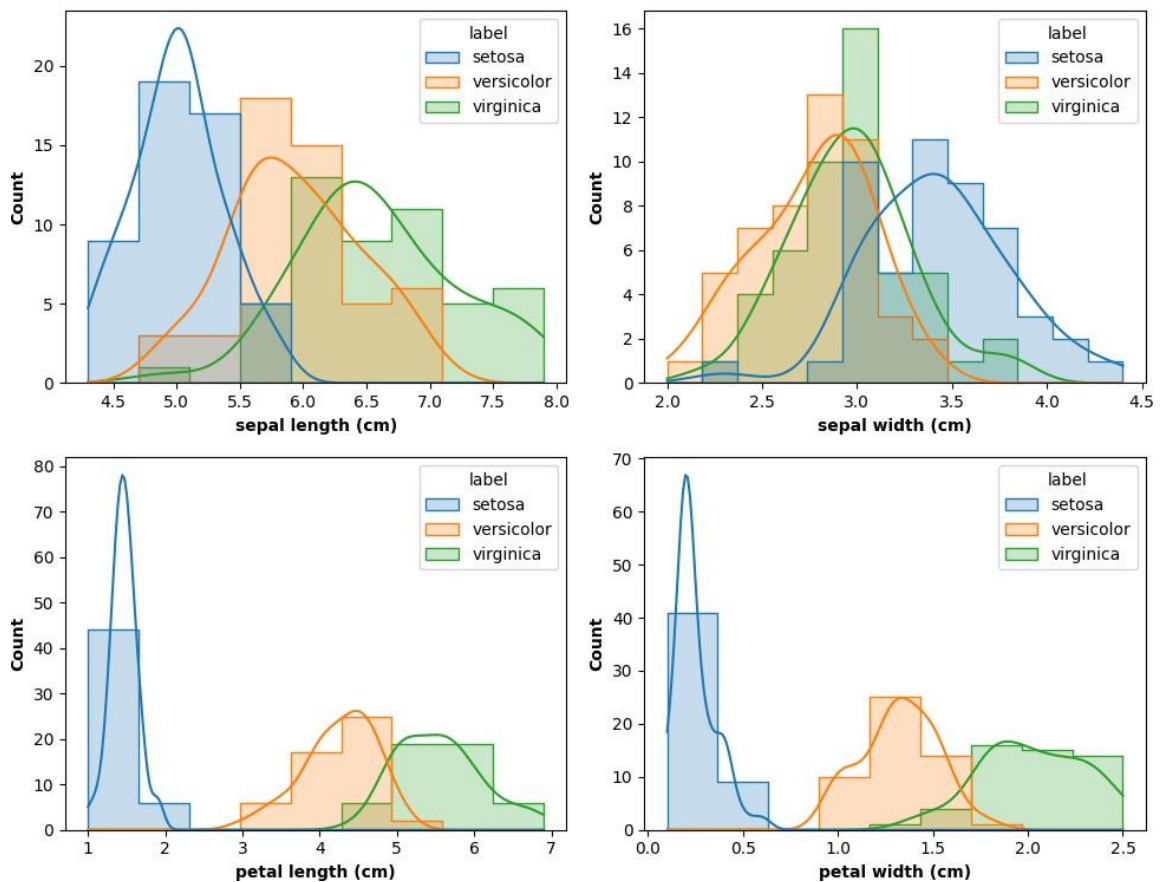
2. Feature Distribution Analysis:

Histograms of each feature were plotted for all three classes.

The plots show that *Setosa* is clearly separable from the other two classes, especially using petal length and petal width.

Versicolor and *Virginica* show partial overlap, indicating a more challenging classification boundary between these two classes.

Feature Distributions by Class

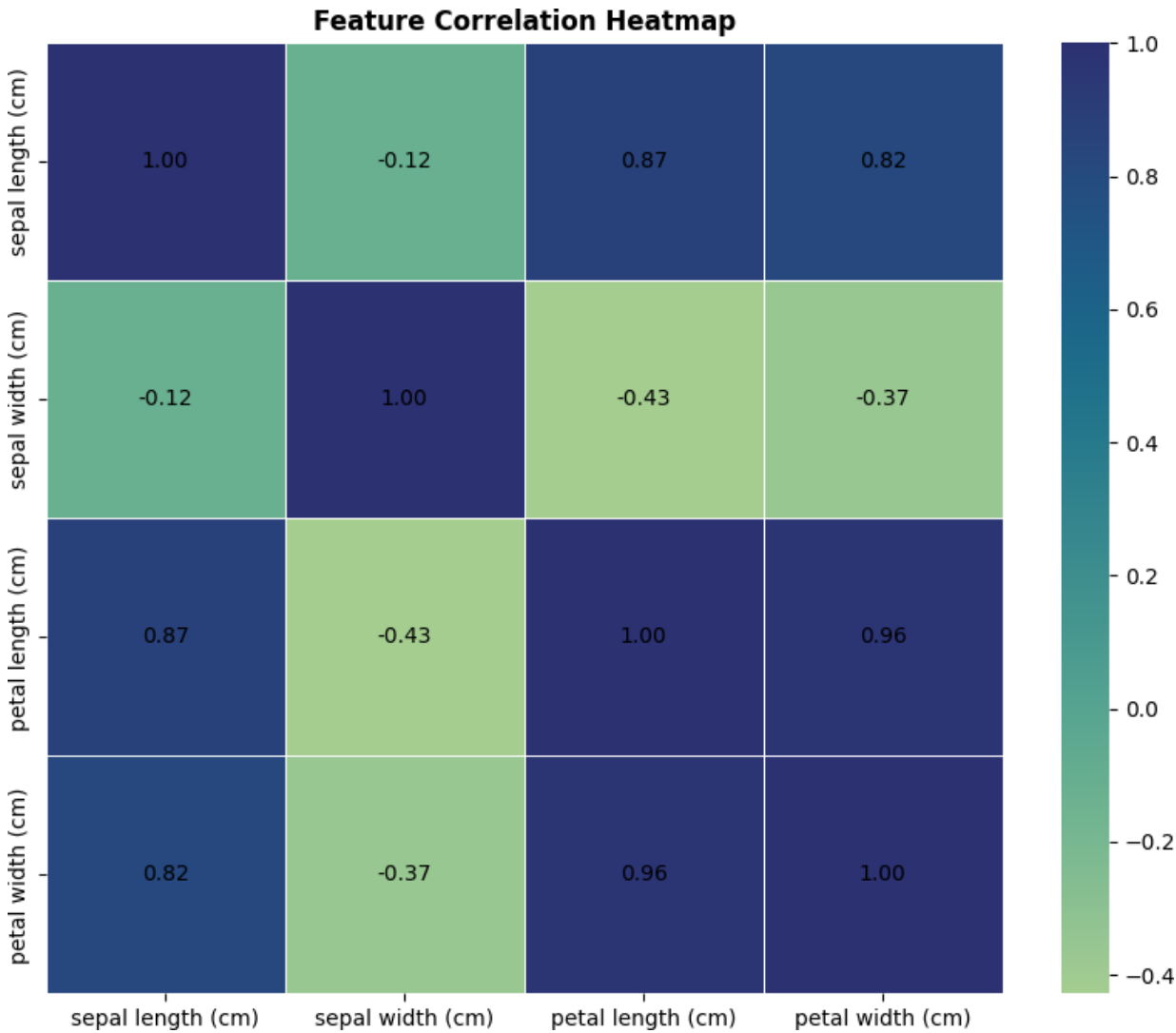


3. Feature Correlation Analysis:

The correlation heatmap indicates a strong positive correlation between petal length and petal width (correlation ≈ 0.96).

Sepal width shows weak negative correlation with petal features.

This analysis suggests that petal measurements are more informative for classification.



The correlation heatmap indicates a strong positive correlation between petal length and petal width (correlation ≈ 0.96).

4. Model Performance (k = 10):

The KNN classifier was trained with **k = 10** using standardized feature values.

Classification Report:

- *Setosa*: Precision = 1.00, Recall = 1.00
- *Versicolor*: Precision = 0.83, Recall = 1.00
- *Virginica*: Precision = 1.00, Recall = 0.80

Overall Accuracy:

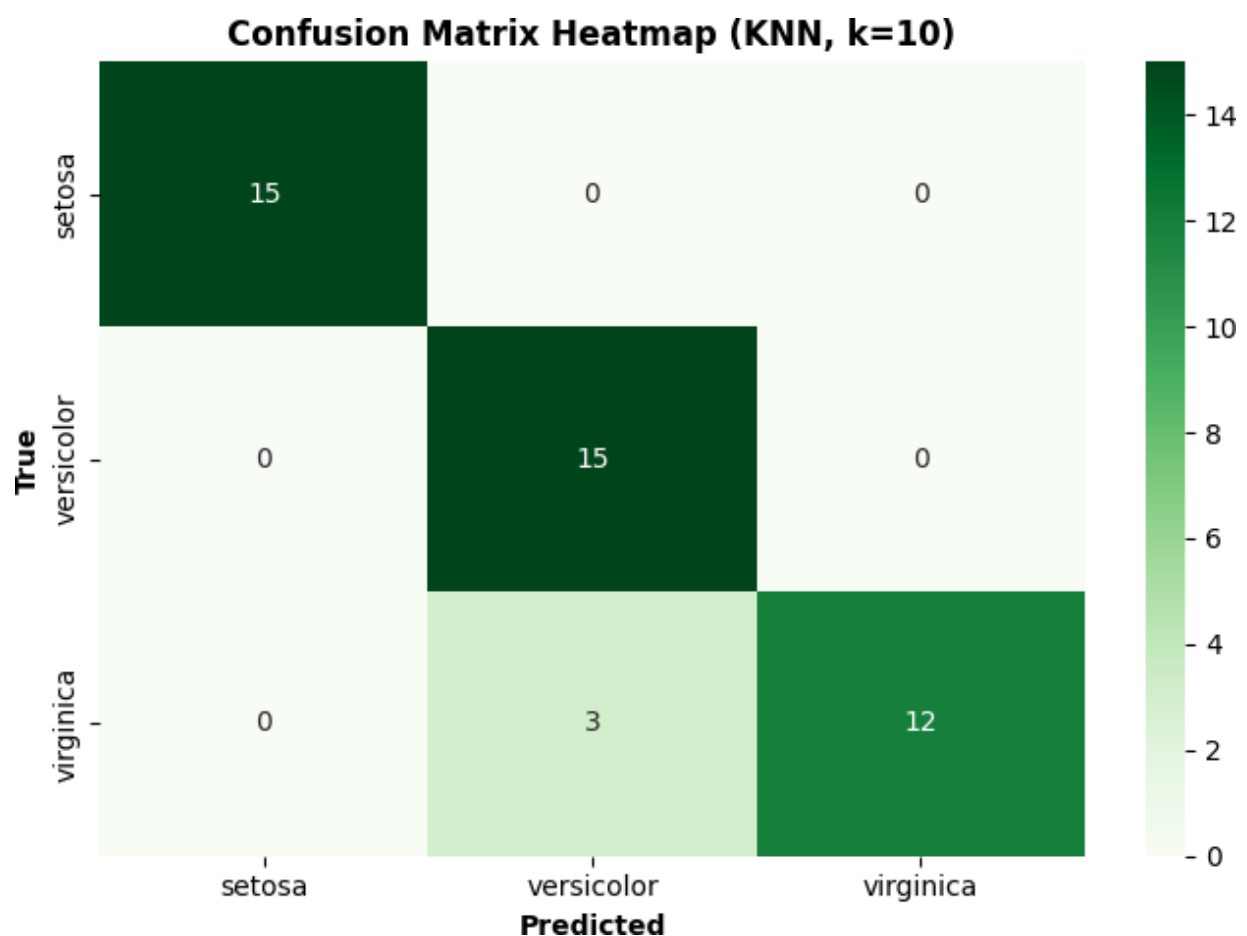
Accuracy = 93.33%

5. Confusion Matrix Analysis:

The confusion matrix shows:

- All *Setosa* samples were correctly classified.
- *Versicolor* samples were classified accurately.
- A few *Virginica* samples were misclassified as *Versicolor*.

This indicates strong model performance with minor confusion between similar classes.



6. Final Evaluation Metrics:

Metric	Value
Accuracy	0.9333
Precision (Macro Avg)	0.9444
Recall (Macro Avg)	0.9333
F1-Score (Macro Avg)	0.9327
ROC-AUC (OVR)	0.9948

The high ROC-AUC score confirms excellent class separability.